An agent is one HTTP request in a loop

Animated explainer · src/llm.js

No SDK, no framework, no hidden retry policy. llm.js sends one OpenAI-compatible request with fetch, turns failures into sentences, and returns the provider response unchanged.

7 scenes · one POST · 90s default timeout · no SDK

CONTENTS

What is in this explainer

01 · Why this file exists 3

02 · The request 4

03 · Timeouts 5

04 · Errors 6

05 · Parsing 7

06 · Sanitising 8

07 · What crosses 9

08 · Recap 10

Page numbers refer to the folio at the foot of each page.

01 · Why this file exists

One request. That is the model layer.

SRC/LLM.JS · *source*

```
const ENDPOINT = 'https://openrouter.ai/api/v1/chat/completions';
const DEFAULT_TIMEOUT_MS = 90_000;

export async function chat({ apiKey, model, messages, tools, signal, timeoutMs }) {
  if (!apiKey) throw new Error('OPENROUTER_API_KEY is not set');
  // ...one fetch...
}
```

Messages — The whole conversation.

Tools — The schemas the model may call.

Timeout — A stalled socket fails.

> **remember** The point of the demo is that an agent is just this HTTP request in a loop.
> **On screen:** This file is the model layer: one endpoint and one exported function. It receives a key, a model, messages, tools, and a timeout. No SDK. A single fetch is enough to make the whole agent work.

02 · The request

What goes over the wire.

SRC/LLM.JS · *source*

```
res = await fetch(ENDPOINT, {
  method: 'POST',
  signal: abort,
  headers: {
    Authorization: 'Bearer ' + apiKey,
    'Content-Type': 'application/json',
    'X-Title': 'min-coding-js',
  },
  body: JSON.stringify({ model, messages, tools, max_tokens: 2048 }),
});
```

method — POST

auth — Bearer key, never logged

body — model, messages, tools, max_tokens

title — a small attribution header

> **On screen:** The request is a normal JSON POST. The key travels in an Authorization header. The body carries the model slug, the message array, the tool schemas, and a token cap. There is no hidden prompting and no SDK magic. What you see is what the provider receives.

03 · Timeouts

A stalled socket must fail.

SRC/LLM.JS · *source*

```
const timeout = AbortSignal.timeout(timeoutMs);
const abort = signal ? AbortSignal.any([signal, timeout]) : timeout;

try {
  res = await fetch(ENDPOINT, { /* ... */ signal: abort });
  text = await res.text();
} catch (err) {
  if (timeout.aborted) {
    throw new Error('request timed out after ' + Math.round(timeoutMs / 1000) + 's');
  }
  throw err;
}
```

AbortSignal.timeout creates the deadline. AbortSignal.any combines it with an optional caller signal.
remember A hung request is worse than a failed request: it has no error to show and no exit code to explain.
On screen: The default timeout is 90 seconds. A stalled provider must not hang the session. AbortSignal.timeout creates the deadline, and AbortSignal.any lets the caller add another abort signal. If the timeout fired, the error says so. If something else happened, the original error is rethrown.

04 · Errors

HTTP status becomes a sentence.

• 401 Bad key “OpenRouter 401 (bad or revoked key): ...”

• 402 No credits “OpenRouter 402 (out of credits): ...”

• 🗒️ Non-JSON “OpenRouter returned non-JSON: ...”

• ⚠️ Provider error “OpenRouter error: ...”

SRC/LLM.JS · *source*

```
if (!res.ok) {
  const hint = res.status === 401 ? ' (bad or revoked key)'
    : res.status === 402 ? ' (out of credits)' : '';
  throw new Error('OpenRouter ' + res.status + hint + ': ' + text.slice(0, 400));
}
```

> **On screen:** The caller should never have to read a raw HTTP body to understand a failure. 401 says bad key. 402 says out of credits. Other statuses carry the response text. Parsing errors and provider errors get their own messages too.

05 · Parsing

Trust nothing until it parses.

SRC/LLM.JS · *source*

```
let json;
try {
  json = JSON.parse(text);
} catch {
  throw new Error('OpenRouter returned non-JSON: ' + text.slice(0, 200));
}
if (json.error) {
  throw new Error('OpenRouter error: ' + JSON.stringify(json.error).slice(0, 400));
}
return json;
```

> The response is a provider envelope. The loop, not this file, decides what choices[0].message means.
> **On screen:** The body is parsed explicitly. A proxy or an error page should not become a mysterious downstream crash. A JSON error field is surfaced as a provider error. Otherwise the parsed response is returned unchanged. This layer stays a transport.

06 · Sanitising

Do not echo the reasoning trace.

SRC/LLM.JS · *source*

```
export function sanitizeAssistant(message) {
  const out = { role: 'assistant', content: message.content ?? '' };
  if (Array.isArray(message.tool_calls) && message.tool_calls.length > 0) {
    out.tool_calls = message.tool_calls;
    if (message.content == null) delete out.content;
  }
  return out;
}
```

- 🧠 Keep role, content, and tool_calls.
- ✂️ Drop reasoning and reasoning_content.
- 🔁 Why otherwise the whole trace is re-sent on every later step.

> **On screen:** Providers may return extra fields, including reasoning traces. sanitizeAssistant rebuilds the assistant message with only the fields an agent needs. Echoing the trace back would re-send it on every later step. The sanitizer prevents that context leak.

07 · What crosses

One clean boundary.

messages + tools

fetch — POST

provider JSON

sanitize — assistant message

> Everything above this file is provider-shaped. Everything below is plain JavaScript objects.
> **remember** The model layer is small on purpose. The agent is the loop around it.
> **On screen:** The boundary is simple: messages and tools go out, a provider envelope comes back. The loop turns that envelope into an assistant message and sanitises it. Keeping this boundary small is what makes the rest of the agent readable.

08 · Recap

One fetch. No hidden layers.

• 1 endpoint

• 90s default timeout

• 0 SDKs

→

One request. No hidden retries or model framework.

→

Fail loudly. Timeout, status, non-JSON and provider errors all say what happened.

→

Return the envelope. The loop owns the meaning of choices[0].

→

Sanitise the assistant. Do not re-send reasoning traces.

> **remember** The model is not a service you integrate. It is a POST request you make.
> **On screen:** One endpoint, a 90-second default timeout, and no SDK. Fail with readable errors, return the envelope, and sanitise the assistant message. That is the whole model layer.